

Qwen 与模拟电路 Multi-Agent 使用手册

项目根目录：/home/xu。部署日期：2026-10-02（日本时间）。本手册区分真实的 GPU 模型推理和用于测试的软件模拟数据。

实际环境

系统：Ubuntu 20.04.5 LTS；4 x NVIDIA RTX A6000，每卡 49140 MiB；驱动 550.144.03。选择 GPU 0，其他三张卡不参与项目。

torch 2.6.0：张量计算与 GPU 运算。

vllm 0.8.5：模型推理与本地 API 服务。

transformers 4.51.3：Qwen 模型结构和分词支持。

huggingface-hub 0.30.2：模型下载。

openai 1.78.1：统一 API 客户端。

reportlab 4.4.1：中文 PDF 生成。

Python 3.11.13：项目独立解释器；uv 0.8.22：下载解释器和安装依赖。Git 复用系统命令；未在用户主目录初始化 Git 仓库。

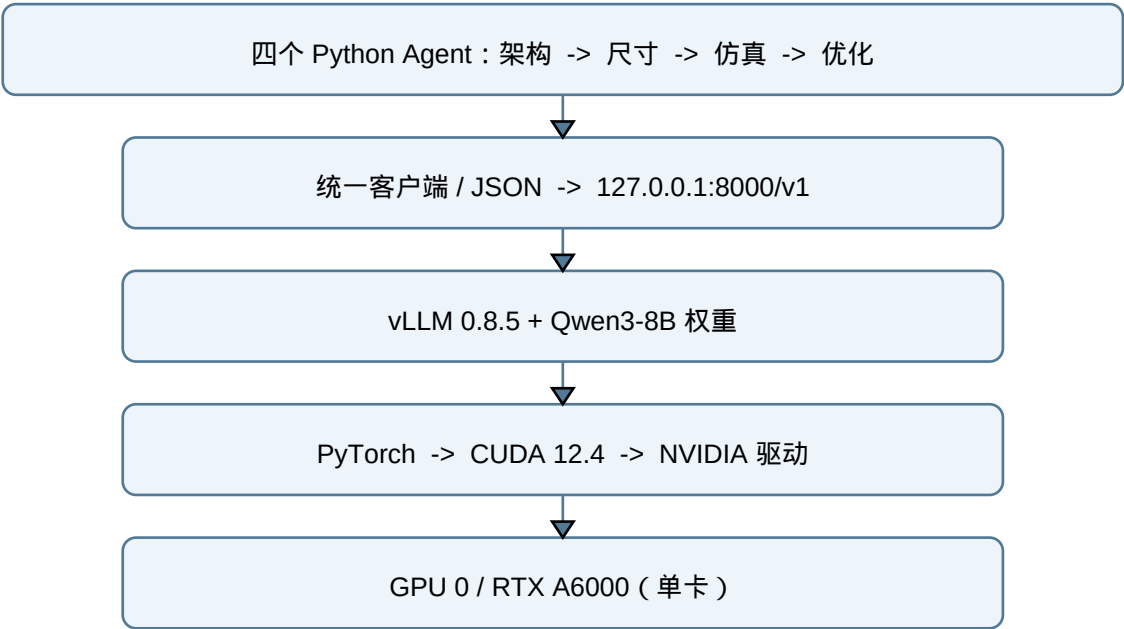
PyTorch 自带 CUDA 12.4 运行库，不依赖系统 nvcc。PATH 中 nvcc 为 CUDA Toolkit 10.1；另发现 /usr/local/cuda-11.8 与 cuda-12.1。未安装或修改系统 CUDA 与 NVIDIA 驱动。

模型：Qwen/Qwen3-8B，BF16，约 16 GB 权重。固定修订版：b968826d9c46dd6066d109eabc6255188de91218

重要限制：未发现可用电路仿真器，且没有工艺 PDK。所有电路指标均为明确标记的 Mock 合成数据，不能证明电路性能，不能作为流片依据。

软件之间的关系

Python 运行各 Agent。Agent 经 OpenAI Python SDK 发送 HTTP 请求给本机 vLLM；vLLM 加载 Qwen 权重，再通过 PyTorch、CUDA 运行库和 NVIDIA 驱动使用 GPU。模型文件本身不是服务，必须先启动 vLLM。



仿真分支：Simulation Agent 选择分析类型 -> 固定模板生成概念网表 -> MockSimulation 返回合成指标 -> Optimization Agent 提出参数更新 -> 再次评估。模型不会执行任意 shell 命令或生成的程序。

当前网表演示 NMOS 差分对、PMOS 电流镜负载、PMOS 共源第二级和 Miller 补偿。偏置采用理想电流源，LEVEL=1 模型仅为占位；生成的 .cir 文件未交给 SPICE 执行。

上下文长度是单次输入与输出 token 的总容量，默认 8192。默认关闭 Qwen 思考模式，以便稳定输出 JSON。JSON 校验只保证格式及参数范围，不保证电路设计正确。

目录与日常命令

/home/xu/	
.venv/	Python virtual environment
.runtime/	Python, caches, temp files
models/Qwen3-8B/	local model weights
config/	settings.json
prompts/	four agent prompts
analog_agents/	client, agents, workflow, tools
scripts/	start, stop, test, install, guide
tests/	offline unit tests
logs/	service and verification logs
outputs/	timestamped experiment runs
docs/	manual source
requirements.lock	complete pinned dependencies
USER_GUIDE.pdf	this manual

在服务器 SSH 终端执行

```
cd /home/xu
source scripts/activate.sh
bash scripts/start.sh
bash scripts/test_qwen.sh
bash scripts/run_demo.sh
bash scripts/stop.sh
```

启动脚本会等待健康检查成功后返回；推理服务在后台运行。stop.sh 仅停止本项目记录的进程组，不会使用 pkill 或清理其他用户进程。重复启动会提示已有服务。

查看日志和 GPU

```
cd /home/xu
tail -n 60 logs/qwen.log
nvidia-smi
cat logs/verification.json
```

在 Codex 沙盒内，GPU 和网络访问可能被隔离，需经过沙盒外执行审批；普通 SSH 终端没有这个工具沙盒限制。服务只绑定本机地址，不直接开放到网络。

修改配置和扩展 Agent

统一配置文件 config/settings.json：gpu 为 nvidia-smi 的卡号；port 为本地端口；context_length 为上下文长度；gpu_memory_utilization 为显存比例，默认 0.5；max_iterations 为最大参数更新次数，默认 2。更改服务参数后必须停止并重新启动服务。

每次启动会重新查询 GPU UUID、显存和利用率。显存已用超过 1024 MiB 或利用率超过 5% 时拒绝启动。此检查不是共享服务器的资源预约；仍需遵守实验室 GPU 调度约定。

替换模型时同时修改 model_repo、model_revision（固定 commit）、model_dir 和 served_model；确认新模型被当前推理版本支持，并检查磁盘及显存。下载不会自动升级软件版本。

```
cd /home/xu
source scripts/activate.sh
python scripts/download_model.py
bash scripts/start.sh
```

修改 prompts/architecture.md、sizing.md、simulation.md、optimization.md 调整角色提示词。修改 specification 调整电压、负载、增益、带宽、相位裕度和功耗目标。所有单位写在 JSON 字段名中。

每次运行都会创建 outputs/时间戳/，保存配置快照、架构、尺寸、history.json、summary.json 和每轮概念网表。失败写入 failure.json 并返回非零状态，禁止把失败运行当成完成。

新增温度传感器或 ADC 时，应同时新增架构提示词、参数 JSON Schema、网表模板和仿真指标。真实仿真工具可实现 simulation.py 的 Simulator 接口，加入受限的工具调用及结果解析，再开放配置；当前仅接受 mock，避免误报真实结果。

当前是原生 Python 顺序编排。每个 Agent 都真实调用本地 Qwen；优化建议经范围检查后应用，再交给仿真接口重新计算。最多进行初始评估加 2 轮参数更新。

验证结果与边界

Python 与依赖：通过：Python 3.11.13，pip check 无依赖冲突

PyTorch GPU 运算：通过：2.6.0+cu124，单卡 RTX A6000，GPU 张量求和结果 4950

Qwen 加载：通过：Qwen3-8B BF16，权重显存约 15.27 GiB，vLLM 健康检查成功

本地 API：通过：127.0.0.1:8000/v1，JSON 算术测试 $19+23=42$

四个独立 Agent：通过：架构、尺寸、仿真、优化分别真实调用本地模型并通过 JSON Schema 校验

端到端与实际迭代：通过：第二次演示执行初始评估与 2 轮迭代，输入管宽度 20 $\rightarrow$ 25 μm ，偏置 40 $\rightarrow$ 35 μA

演示设计目标：未达成：MOCK 增益 60.612 $\rightarrow$ 61.775 dB，仍低于 65 dB；其余 MOCK 目标满足。不得作为真实电路指标

单元测试：通过：4 项，包括路径越界、非法参数拒绝、合成数据标记和更新参数生效

项目路径检查：通过：缓存/临时路径均位于项目内，生成目录中未发现指向项目外的符号链接；此项不是内核级全部系统调用审计

GPU 隔离与停止：通过：仅 GPU 0 推理（采样利用率 95%，显存 24064 MiB）；停止后四卡均恢复 5 MiB，未发现计算进程

测试证据：logs/gpu_test.json、api_test.json、agent_tests.json、unit_tests.log、dependency_check.log、gpu_during.csv，以及 outputs/ 下的运行历史。实际失败及解决过程记录在 logs/verification.json。

模型推理通过不代表电路性能达标。尚未完成：真实 SPICE、工艺 PDK、偏置电路完整设计、PVT/失配验证、版图和流片验证。Mock 的数学公式是教学用合成规则，不是真实晶体管模型。

离线测试、独立 Agent 测试与重建文档

```
cd /home/xu
source scripts/activate.sh
python -m pytest -q --basetemp="$TMPDIR/pytest"
python scripts/test_agents.py
python scripts/build_guide.py
```

可重复安装

```
cd /home/xu
bash scripts/install.sh
```

install.sh 使用完整锁定依赖 requirements.lock 和固定模型修订版。需要联网，磁盘应留出足够空间。现有环境可直接复用，无须每天重复安装。缓存、临时文件、下载和日志均由 scripts/env.sh 指向项目内目录；不要绕过脚本直接运行下载或服务。

常见问题与排查

无法识别 GPU

先运行 `nvidia-smi`。若 SSH 中可用而工具沙盒中不可用，是沙盒隔离；通过审批执行 GPU 操作。若 SSH 本身也失败，应联系管理员，不要自行升级共享服务器驱动。

服务启动失败或端口冲突

查看 `logs/qwen.log`。端口占用时改 `config/settings.json` 的 `port`；GPU 忙碌时等待或选择一张空闲卡，不要终止其他用户作业。缺少模型时先运行下载脚本。

显存不足、响应超时或 JSON 截断

减小 `context_length` 或请求长度；在共享卡上不要盲目增大显存占用。输出截断可适当提高 `max_tokens`，但输入和输出总量必须小于上下文长度。超时可检查服务日志并调整 `timeout_seconds`。

下载中断或磁盘不足

重新执行 `download_model.py` 可续传。用 `df -h /home/xu` 检查空间。不要删除用户已有文件；只在确认后清理本项目已知缓存。安装记录位于 `logs/install.log`，下载记录位于 `logs/download.log`。

工具运行报依赖错误

确认已 `source scripts/activate.sh`，然后执行 `python -m pip check`。不要使用系统 `pip` 或 `sudo`。依赖升级会破坏已验证组合，应在单独项目环境测试。

参考资料

Qwen 模型官方说明：<https://huggingface.co/Qwen/Qwen3-8B>

vLLM 0.8.5 安装说明：https://docs.vllm.ai/en/v0.8.5/getting_started/installation/gpu.html

PyTorch 历史版本：<https://pytorch.org/get-started/previous-versions/>

本项目以实际安装版本和 GPU 测试为准。未修改 `.bashrc`、`.profile`、系统 Python、系统驱动或其他用户文件。